SOFTWARE ARCHITECTURE & TECHNICAL DESIGN

Bandly — The Artist Operating System

A modular-monolith blueprint for a Next.js + Spring Boot + PostgreSQL platform that gives independent artists a career backoffice — release planning, positioning, booking, rights, finance, and focus — built around one durable idea: a versioned *Artist DNA* that every generated document draws from.

SOURCE MATERIAL
BANDLY Phase 1 & Phase 2 (NL)

STACK
Next.js · Spring Boot · PostgreSQL

STYLE
Modular monolith, DDD-lite

STATUS
Pre-development blueprint

§01

Executive Summary

Bandly is not a marketplace and not a social network. It is a single-tenant, AI-assisted **career backoffice** for an artist or band who has talent but no team — the tool that stands in for a manager, publicist, bookkeeper, and personal assistant until the artist can afford one. Every one of the twelve problem areas in the source material (positioning, planning, content, releases, rights, booking, network, finance, grants, communication, files, focus) is a facet of the same job: absorb the administrative and creative-adjacent load so the artist's limited free time goes to the art.

The architectural center of gravity is the **Artist Identity Profile → Artist DNA** pipeline: a structured onboarding profile that the system compiles into a reusable brand identity (bios, tone-of-voice, pitch language), which every other module then references so a press release, a grant application, and an Instagram caption all sound like the same artist without the artist re-explaining themselves each time.

We recommend a **modular monolith** — one Spring Boot deployable organized into clearly bounded, DDD-lite modules, one PostgreSQL database, one Next.js frontend — with an internal outbox/event mechanism so modules react to each other (a release date sets a checklist; a checklist item due soon triggers a reminder) without service-to-service network calls. Nothing in the source material justifies microservices, a message broker, or a dedicated search engine on day one; all three are named as explicit future options with a stated trigger condition, not defaults.

> READING THIS DOCUMENT
>
> Sections are numbered because they build on each other — the domain model in §04 is the input to every later decision, and the decision records in §20 are the ones that most determine whether §22's growth phase is cheap or expensive to reach. §02 also documents a material correction to the brief this blueprint was requested against; read it before the rest.

§02

Product Understanding

What Bandly is, who it's for, and a correction to a load-bearing assumption in the original brief.

FINDING — THE "PHASE 1 / PHASE 2" SPLIT DOES NOT EXIST IN THE SOURCE MATERIAL

Both PDFs cover the *identical* twelve problem areas. "BANDLY Phase 1" (*De doelgroep begrijpen* — understanding the target audience) frames each area as problem → how AI could help. "BANDLY Phase 2" (*De kansen inschatten* — assessing the opportunity) restates the same twelve areas as input → generated output, plus competitive positioning against ChatGPT, Canva, Notion, DistroKid, Spotify for Artists, Mailchimp, and spreadsheets. Neither document withholds functionality for a later release. There is no phased *feature* scope to architect around.

This blueprint therefore treats all twelve areas as one MVP scope (per your direction) and reserves the terms "Phase 1" and "Phase 2" for something more useful architecturally: what ships first, and what the platform grows into afterward. See §21–§22.

What problem it solves

Emerging artists — musicians, songwriters, producers, and the visual collaborators around them — usually have a day job, real talent, and no professional network to offload the non-creative work onto. The source material is explicit that the danger is asymmetric: AI automating the *art* feels threatening, but AI absorbing the *surrounding administration* is relief. That single sentence is a hard product constraint, not color commentary — see the risk entry in §24.

Who it's for

Primary — the artist / band owner

Sets up the Artist DNA, owns the band workspace, invites collaborators, holds billing and settings. Explicitly *not* a manager, label, or agency — the product's own positioning

Secondary — the band member / collaborator

Inferred, not stated as a named role: the source material calls out task division "within bands or teams" and warns that agreements "disappear into group chats." This implies a

line is "your career manager before you can afford one."

shared workspace with more than one login, not a solo-only tool.

OPEN ARCHITECTURAL QUESTION

No persona is defined beyond the artist/band. There is no venue, label, curator, or fan account — those parties are always the *target* of an artifact the system generates (a pitch email, a press kit), never a platform user. If a future venue-facing or fan-facing surface is wanted, that is new product scope, not an extension of what's documented — flag it explicitly rather than let it creep in through "booking" or "network" features.

Core workflows

1. **Identity, once:** complete the Artist Identity Profile → generate Artist DNA (bios, tone-of-voice, pitch language, EN translation).
2. **Plan a release:** enter title/date/genre/artwork → get a generated checklist, metadata, pre-save copy, press release, playlist pitch, and a post-release evaluation slot.
3. **Book a show:** log a venue/promoter contact → generate a pitch, track the reply, get reminded to follow up, produce a rider and setlist when confirmed.
4. **Keep the books straight:** attach split sheets, credits, and income to a track; get plain-language contract summaries and invoice reminders.
5. **Stay inside the week:** a bounded weekly focus view that surfaces what matters now and explicitly parks what can wait — the one workflow whose job is to reduce, not add, system-generated pressure.

Explicit vs. inferred

EXPLICIT IN SOURCE	INFERRED BY THIS BLUEPRINT
The 12 problem/solution domains	Concrete entities, fields, and relationships within each domain
Artist DNA as a reusable identity base	DNA as a *versioned*, queryable database entity other modules join against
Bands/teams divide tasks	Multi-user org model with owner/member roles

EXPLICIT IN SOURCE	INFERRED BY THIS BLUEPRINT
AI drafts content; specialist review needed for legal/tax	A draft/approved status lifecycle on all generated content
Competing tools listed (DistroKid, Spotify for Artists, Mailchimp…)	These become future integration targets, not Phase 1 dependencies

§03

Requirements Model

The twelve source domains, grouped by architectural weight rather than document order.

Core — the platform doesn't function without these

- **Identity & workspace** — accounts, band membership, roles.
- **Artist DNA** — the identity pipeline every other module depends on.
- **AI generation kernel** — the shared capability that turns DNA + module context into drafts.
- **Media & files** — every domain attaches audio, artwork, or documents to something.
- **Planning & focus** — tasks, deadlines, and the weekly view that ties every module's due dates together.

Supporting — where the product's value is actually felt

- Content Studio (social captions, scripts, calendars)
- Release Manager (releases, tracks, checklists, press assets)
- Rights & Admin (split sheets, contracts, invoices, income)
- Relationships / CRM (booking pipeline + network contacts — see §06 for why these merge into one module)
- Finance (budgets, break-even, pricing)
- Grants & Funding (project plans, applications)

Future / optional — should not shape the initial architecture

Third-party distribution sync (DistroKid, Spotify for Artists API), real-time chat between platform users, native mobile apps, multi-band-per-user portfolios, marketplace/discovery of other artists or venues, payments between users. Each is addressed as a deliberate extension point in §22, not designed in from day one.

Feature breakdown

FIG. 1 — FEATURE-TO-WORKFLOW MAP

DOMAIN	USER	TRIGGER	KEY DATA	NOTABLE BUSINESS RULE
Positioning	Owner	Onboarding / DNA refresh	Identity profile answers, references, tone words	DNA is versioned; downstream drafts pin to a DNA version
Planning	Owner, member	Any deadline created elsewhere	Tasks, deadlines, weekly plan	Weekly view caps visible items — never lists everything at once
Content Studio	Owner, member	Voice memo / text input, or release event	Content ideas, captions, calendar slots	Requires an approved DNA before first generation
Release Manager	Owner	New release drafted	Release, tracks, metadata, checklist	Creating a release auto-spawns its checklist tasks
Rights & Admin	Owner	Track finalized	Split sheet, credits, ISRC, contracts	AI output is always labeled "review with a specialist"
Booking (Relationships)	Owner, member	New venue contact	Contact, pitch, rider, interaction log	Every outbound pitch schedules its own follow-up reminder
Network (Relationships)	Owner	New industry contact	Contact, notes, intro email	Shares the same Contact entity as booking, typed differently
Finance	Owner	Manual entry or	Budget, expense,	Break-even figures recompute on any

DOMAIN	USER	TRIGGER	KEY DATA	NOTABLE BUSINESS RULE
		linked income record	income, scenario	linked change
Grants	Owner	New application started	Project plan, budget, motivation text	Reuses DNA + Finance budget as generation input
Communication	Owner, member	Any module needing outbound text	Drafts, summaries	Cross-cutting capability of AI Generation, not its own aggregate
Files	Owner, member	Any upload	Media asset, per-track structure	Missing-asset warnings surface on the release checklist
Focus	Owner, member	Weekly cycle	Priority list, reflection prompts	Never generates new tasks — only reorders/hides existing ones

§04

Domain Model

Concepts derived from the two documents — not the generic marketplace vocabulary (venue accounts, applications, matching) that a booking platform would use.

FIG. 2 — CORE ENTITIES

ENTITY	PURPOSE	OWNED BY	LIFEC
Band	The workspace / tenant boundary — solo artist or group	Owner user	Root every tenar
BandMember	User's membership + role in a Band	Band	owne mem
IdentityProfile	Raw onboarding answers (who, sound, audience, references)	Band	Freef versi
ArtistDNA	Compiled brand identity: bios, pitch, tone-of-voice, EN translation	IdentityProfile	Gene one " a tim
GeneratedAsset	Any AI-produced document (caption, press release, grant text…)	Band, via a source module	draft appr attrib DNA
Release	A record/single/EP being brought to market	Band	draft relea
Track	A song within a release, or standalone	Release (optional)	Carri ISRC
MediaAsset	A stored file: master, stem, artwork, rider, video	Track, Band, or GeneratedAsset	Obje + typ
SplitSheet	Songwriter/producer share agreement for a track	Track	Perce to 10 signe
Contract / Invoice	Legal/financial documents tied to a track or booking	Track or Contact	AI or neve bindi
Contact	An external party: venue, promoter, label, media, curator	Band	Type `cont` platf

ENTITY	PURPOSE	OWNED BY	LIFEC
Interaction	Logged touchpoint with a Contact (email sent, reply, call)	Contact	Feed remi
Booking	A pitch/negotiation/confirmed show with a venue Contact	Contact	pitch confi
Task	Any actionable item, band- or member-scoped	Band, optionally a source module	Can l spaw Relea event
Budget / Expense / IncomeRecord	Financial planning and actuals	Band, optionally a Track/Release	Incor a Tra profi
GrantApplication	A funding/subsidy application in progress	Band	Reus Finar gener
GenerationJob	An async AI generation request + its result	Band, source module	queu done prom for au
Notification	An in-app/email nudge derived from a Task or Interaction due date	BandMember	Rate- mem §24 r press

§05

Architecture Overview & Style

Recommendation: modular monolith. One Spring Boot application, one PostgreSQL database, organized into the bounded contexts from §04, communicating in-process through well-defined module APIs and a lightweight internal event mechanism (§12) rather than HTTP or a broker.

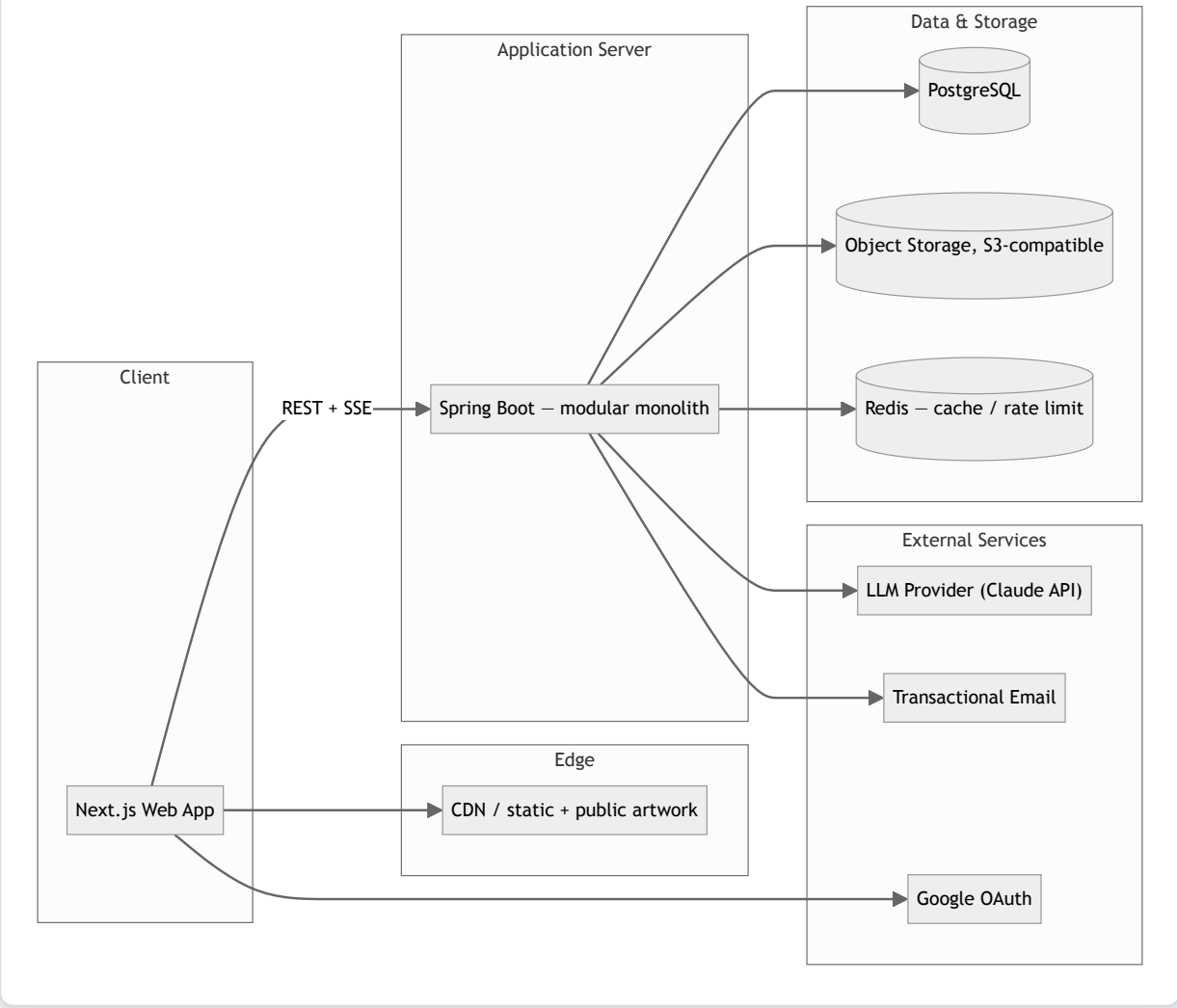
Why not microservices

Bandly's workflows are *transactionally cohesive*: creating a release touches Release Manager, spawns Planning tasks, and may kick off AI Generation and Media — inside one user action. Splitting these into services on day one buys distributed-transaction complexity (sagas, eventual consistency, service discovery, per-service deploys) for a team and load profile that don't need it yet. The source material describes a single-tenant tool used by one artist or small band at a time — not a platform serving heterogeneous, independently-scaling workloads.

Why modular, not a plain layered app

A single flat `service/` package is the failure mode this avoids. Each bounded context owns its own package, its own tables (by naming convention, not separate schemas), and exposes a narrow application-layer interface to the rest of the app. That boundary is what makes later extraction (§22) a refactor, not a rewrite.

FIG. 3 — SYSTEM ARCHITECTURE



§06

Backend Architecture

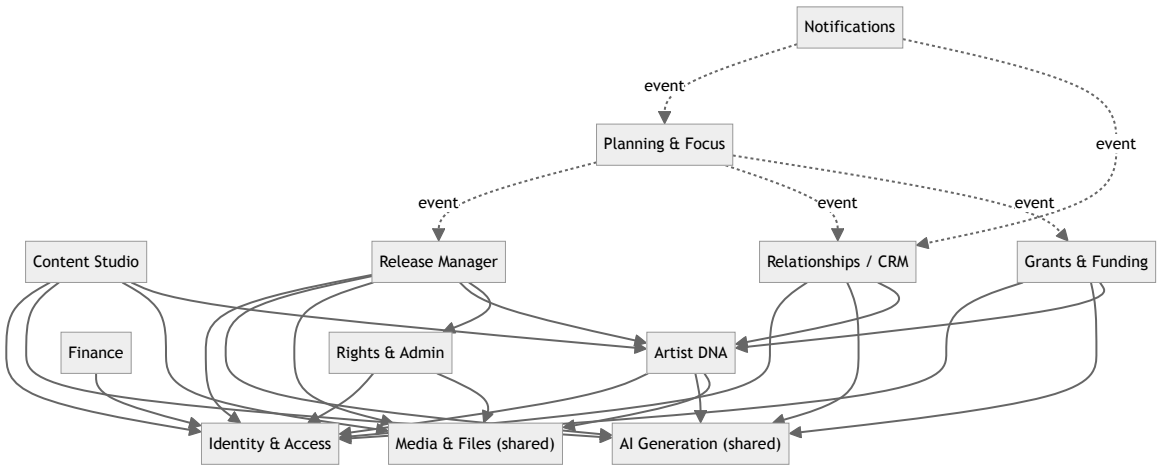
Each module follows a light hexagonal shape: a `domain` package with no framework dependencies, an `application` package holding use cases, and an `infrastructure` package with Spring Data repositories and adapters (LLM client, object storage client). REST controllers are a thin `api` package per module — never a god `controllers/` folder shared across the app.

FIG. 4 — REPOSITORY LAYOUT

```
backend/
├── identity/            # users, bands, membership, auth
├── artistdna/            # identity profile intake, DNA generation + versioning
├── aigeneration/          # shared kernel: prompt templates, job queue, LLM adapter
├── media/                 # shared kernel: object storage, per-track asset graph
├── planning/              # tasks, weekly focus, deadline aggregation
├── content/               # social captions, scripts, content calendar
├── release/               # releases, tracks, checklists, evaluation
├── rights/                # split sheets, contracts, invoices, income
├── relationships/         # contacts, interactions, bookings (venues + network)
├── finance/                # budgets, expenses, scenarios, pricing
├── grants/                 # applications, project plans
├── notifications/          # in-app + email delivery, reminder scheduling
├── platform/                # outbox, event bus, correlation ids, shared config
└── BandlyApplication.java
```

Two modules are deliberately infrastructural rather than "business" domains: **AI Generation** (prompt orchestration + DNA-context injection + async job handling, used by DNA, Content Studio, Release Manager, Relationships, and Grants) and **Media** (object storage + per-track asset graph, used by DNA, Content Studio, Release Manager, and Rights). Everything else depends on these two but not on each other directly — cross-module reactions travel as events (§12), never as one module calling another's repository.

FIG. 5 — MODULE DEPENDENCY GRAPH



Why Booking and Network merge into one Relationships module

The source material describes near-identical mechanics for "boeken van optredens" (venues, promoters) and "netwerk opbouwen" (labels, curators, media): log a contact, send an introduction, follow up, record the reply, get reminded. Splitting these into two modules would duplicate the same Contact/Interaction/Reminder machinery twice for a distinction that's really just a `contact_type` value.

Why Communication is not its own module

"Communicatie en professionaliteit" reads as a capability every other domain needs (a booking pitch, a grant motivation, a press mail are all generated text), not a domain with its own entities. It's implemented as the AI Generation module's output surface, plus a small `Interaction` log already owned by Relationships for anything that needs a durable record of what was sent and when.

§07

Database Architecture

One PostgreSQL database, tables prefixed by module (`release_`, `rights_`, `rel_`…) rather than separate schemas — keeps cross-module reporting queries simple while the module boundary stays a code-level, not database-level, concern.

Relational vs. JSONB

- **Relational:** anything queried, filtered, or joined — releases, tracks, tasks, contacts, bookings, budgets, split-sheet percentages. This is the large majority of the schema.
- **JSONB:** two deliberate exceptions — (1) `identity_profiles.answers`, because onboarding questions may evolve and the raw Q&A is read as a whole, not filtered field-by-field; (2) `generation_jobs.prompt_payload` / `response_payload`, an audit trail of what was sent to and returned from the LLM, never queried structurally.

REPRESENTATIVE TABLES

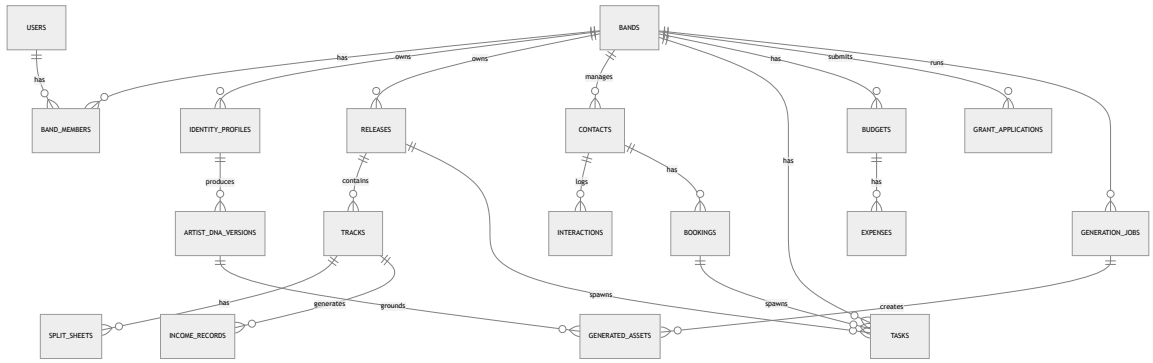
FIG. 6 — KEY TABLES

TABLE	KEY FIELDS	NOTES
bands	id, name, created_at, plan_tier	Tenant root
band_members	band_id, user_id, role, invited_at	Unique (band_id, user_id); role enum
artist_dna_versions	id, band_id, identity_profile_id, status, created_at	One row per generation; `current_version_id` pointer on band
generated_assets	id, band_id, dna_version_id, source_module, asset_type, content, status	status: draft / edited / approved; indexed on (band_id, asset_type)
releases	id, band_id, title, release_date, genre, status	status drives checklist task generation
tracks	id, release_id (nullable), isrc, credits_locked	credits_locked freezes split_sheets
split_sheets	track_id, contributor_name, role, percentage	CHECK: sum(percentage) = 100 per track (app-enforced + DB trigger)
contacts	id, band_id, contact_type, name, org, email	contact_type: venue / promoter / label / media / curator / other
bookings	id, contact_id, status, proposed_date, gage_estimate	status: pitched / replied / confirmed / played / declined
tasks	id, band_id, assignee_id,	source_type/source_id link back to Release,

TABLE	KEY FIELDS	NOTES
	source_type, source_id, due_at, done_at	Booking, Grant, or null (manual)
budgets / expenses / income_records	band_id, track_id (nullable), amount, category, occurred_at	income_records optionally link a track for per-song P&L
grant_applications	id, band_id, scheme_name, status, budget_id	status: drafting / submitted / decided
generation_jobs	id, band_id, module, status, prompt_payload (jsonb), response_payload (jsonb)	Async job + audit record; see §12
notifications	id, member_id, task_id (nullable), channel, sent_at, read_at	Rate-limited at insert time, not just at send time

All primary tables carry `created_at` / `updated_at`; financially or legally material tables (`split_sheets`, `contracts`, `income_records`) use soft-delete (`deleted_at`) rather than hard delete, since these back audit and, eventually, tax-relevant history. Everything else hard-deletes.

FIG. 7 — ENTITY RELATIONSHIPS



§08

API Architecture

REST, versioned under `/api/v1`, band-scoped in the path since almost nothing in Bandly exists outside a band's tenancy boundary. OpenAPI generated via springdoc; the frontend's typed client is generated from it, not hand-maintained.

FIG. 8 — REPRESENTATIVE ENDPOINTS

```
POST   /api/v1/bands/{bandId}/identity-profile
POST   /api/v1/bands/{bandId}/dna/generate
GET    /api/v1/bands/{bandId}/dna/current

POST   /api/v1/bands/{bandId}/releases
GET    /api/v1/bands/{bandId}/releases/{releaseId}
POST   /api/v1/bands/{bandId}/releases/{releaseId}/tracks
GET    /api/v1/bands/{bandId}/releases/{releaseId}/checklist

POST   /api/v1/bands/{bandId}/tracks/{trackId}/split-sheets
POST   /api/v1/bands/{bandId}/tracks/{trackId}/assets       # presigned upload

POST   /api/v1/bands/{bandId}/contacts
POST   /api/v1/bands/{bandId}/contacts/{contactId}/bookings
POST   /api/v1/bands/{bandId}/bookings/{bookingId}/pitch     # triggers AI Generation

GET    /api/v1/bands/{bandId}/tasks?due_before=...
PATCH  /api/v1/tasks/{taskId}

POST   /api/v1/bands/{bandId}/grants
GET    /api/v1/bands/{bandId}/finance/budgets

POST   /api/v1/bands/{bandId}/generation-jobs               # async, SSE status
GET    /api/v1/bands/{bandId}/generation-jobs/{jobId}/events # text/event-stream
```

- **Pagination:** cursor-based on high-volume feeds (tasks, interactions, notifications); offset/limit elsewhere.
- **Idempotency:** mutation endpoints that trigger AI generation accept an `Idempotency-Key` header — LLM calls cost money, and a retried click must not double-bill or double-draft.
- **Errors:** RFC 7807 `application/problem+json`, consistent across modules.

- **File upload:** client requests a presigned URL from `/assets`, uploads directly to object storage, then confirms — the API server never proxies audio/artwork bytes.
- **Rate limiting:** per-band token bucket on generation endpoints specifically (cost control, see §11), separate from general API rate limits.

§09

Frontend Architecture

Next.js App Router, TypeScript, organized by the same bounded contexts as the backend so a developer working on "releases" never has to guess which folder owns it.

FIG. 9 — FRONTEND STRUCTURE

```
frontend/
├── app/
│   ├── (auth)/login, register, reset-password
│   └── bands/[bandId]/
│       ├── dna/                 # identity profile + DNA review
│       ├── releases/[releaseId]/
│       ├── content/
│       ├── rights/
│       ├── relationships/       # bookings + network, tabbed
│       ├── finance/
│       ├── grants/
│       └── focus/                # weekly dashboard, default landing page
├── modules/                       # one folder per bounded context
│   ├── releases/{api,components,hooks,types}
│   ├── relationships/{...}
│   └── dna/{...}
├── shared/
│   ├── ui/                        # design-system primitives only
│   ├── api-client/                # generated from OpenAPI
│   └── auth/
└── middleware.ts                    # route-level auth guard
```

- **Server Components** for read-heavy views (release detail, contact list, budget dashboard) — fetch straight from the API server-side, no client waterfall.
- **Client Components** only where interaction demands it: the DNA editor, generation-review panels (accept/edit/regenerate a draft), and the SSE-driven generation progress view.
- **Generated content review** is a first-class UI pattern, not a chat window: every AI draft renders as an editable document with a visible *draft → approved* state, matching the "AI drafts, artist decides" principle from §02.

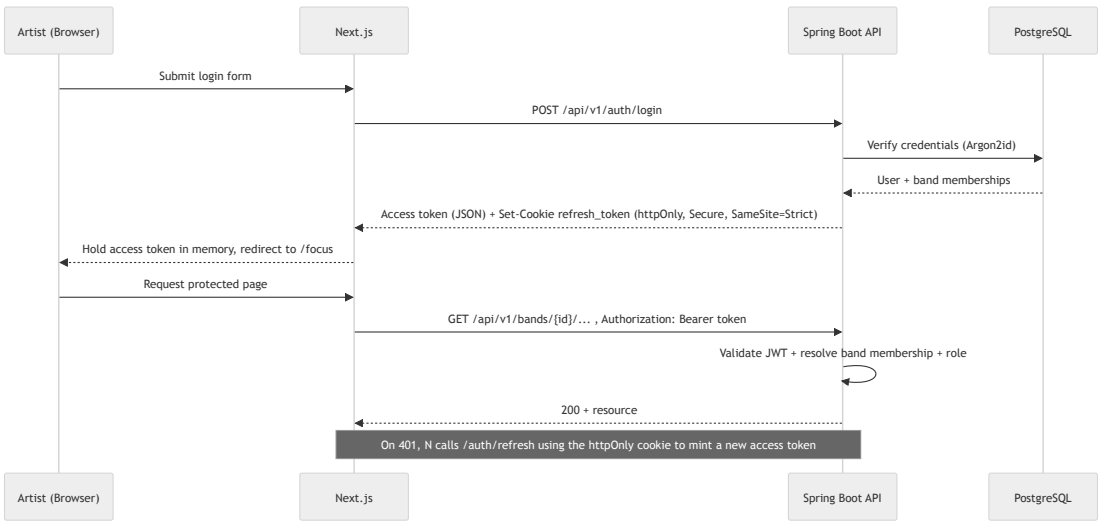
- **State:** server state via the generated API client + React Query-style caching; local UI state (form drafts, focus-view filters) via component state — no global client store needed at this scope.

§10

Authentication & Authorization

- Email/password (Argon2id hashing) + optional Google OAuth.
- Short-lived JWT access token (≈15 min), held in memory on the client — never localStorage.
- Refresh token as an httpOnly, Secure, SameSite=Strict cookie; rotated on use.
- Email verification required before a band can invite additional members.
- Authorization is band-scoped RBAC: every request resolves `(user, bandId) → role` via membership lookup; role gates mutation endpoints (e.g., only owner/admin can invite, only owner sees billing).

FIG. 10 — LOGIN AND AUTHORIZED REQUEST



§11

Security Architecture

Standard OWASP surface

Parameterized queries throughout (Spring Data), CSRF mitigated by SameSite cookies + double-submit token on state-changing forms, strict CORS allow-list, CSP headers, dependency scanning in CI (Dependabot/Snyk).

Generation cost as an attack surface

LLM calls are billable; the per-band token-bucket rate limit from §08 is a security control as much as a cost control — without it, a compromised session or scripted abuse becomes a direct financial drain.

Upload safety

Audio/artwork/document uploads are type- and size-validated server-side before a presigned URL is issued, and scanned (ClamAV or a vendor scanning API) before being served back publicly.

Rights & finance data

Contracts, invoices, and split sheets are the one place the source material explicitly asks for caution ("juridische of fiscale beslissingen moeten door een specialist gecontroleerd worden"). These tables are audit-logged and every AI-generated summary carries a persistent "not legal/tax advice" flag in the data model, not just the UI copy.

DATA PROTECTION

Dutch/EU user base implies GDPR by default even though the source material doesn't name it: right to export and delete, data processing agreement with the LLM provider, and explicit user consent for any use of uploaded content (identity answers, voice memos, lyrics) beyond generating the artist's own drafts.

OPEN ARCHITECTURAL QUESTION

No LLM provider or data-handling policy is specified in the source material. This blueprint assumes a provider offering a zero-retention / no-training-on-customer-data commercial agreement (e.g., Claude API under Anthropic's commercial terms) — confirm before building, since split sheets, contracts, and identity profiles are sensitive content to send to a third-party model.

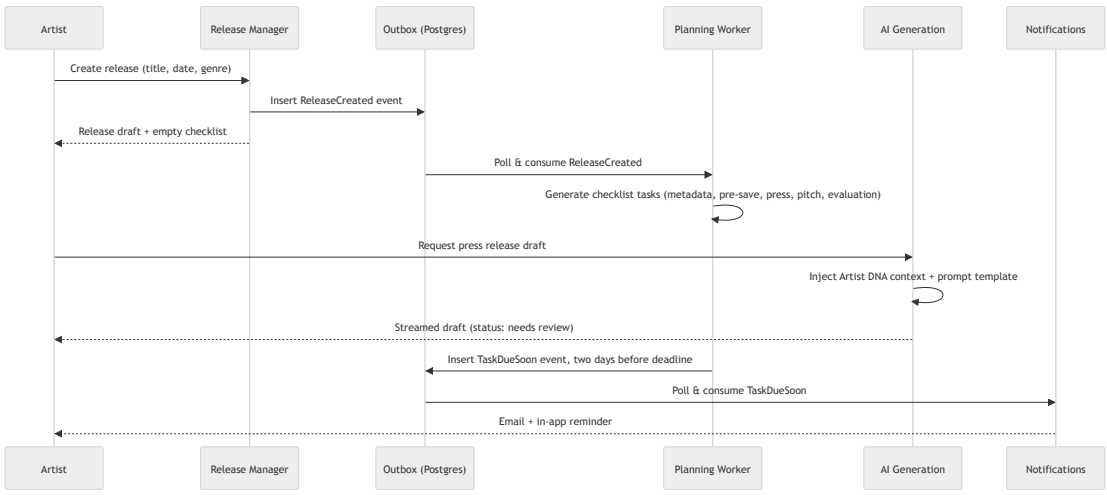
§12

Events & Asynchronous Processing

No message broker. A **transactional outbox table in PostgreSQL**, written in the same transaction as the domain change, polled by a small in-process worker that dispatches to subscribing modules. This gets the decoupling benefit (Planning doesn't need to know Release Manager exists to react to a release date) without the operational cost of Kafka/RabbitMQ — which nothing in the source material justifies at this scale.

AI generation itself is naturally asynchronous: a `generation_jobs` row moves `queued → running → done/failed`, with progress streamed to the browser over Server-Sent Events rather than long-polling or a websocket, since it's one-directional server→client progress, not bidirectional chat.

FIG. 11 — RELEASE CREATION, END TO END



Migration path: if generation volume or cross-module fan-out grows past what a single poller comfortably handles, the outbox table is already the durable log a real broker would read from — swapping the poller for a Debezium/Kafka Connect pipeline is additive, not a rewrite.

§13

Media & File Architecture

Directly answers "bestandsbeheer en workflow" — the source material's own framing is a *logical* structure per track (audio, stems, artwork, lyrics, credits, promo, video, admin all findable from one place), which this blueprint implements as a data model, not a literal folder hierarchy.

- S3-compatible object storage (AWS S3, Cloudflare R2, or equivalent); every `MediaAsset` row is a pointer, never a local disk path.
- Presigned upload/download URLs — the API server issues and verifies, never proxies bytes.
- Public assets (approved artwork, press photos) served through a CDN; masters, stems, and unreleased material stay behind short-lived signed URLs.
- Automatic naming convention (`{band}/{track}/{asset_type}/{version}`) enforced at upload time — directly implements "automatische bestandsnamen voorstellen."
- The Release checklist cross-references expected asset types per track and flags gaps — implements "waarschuwen wanneer belangrijke bestanden ontbreken" as a real system behavior, not a manual review step.
- Soft-delete with a retention window before permanent purge; supports the GDPR export/delete flow in §11.

§14

Search & Discovery

This is the section where the mismatch flagged in §02 matters most: because Bandly has no artist-to-artist or artist-to-venue matching, there is no "search and matching engine" to design. What the domain actually needs is in-workspace lookup — finding a contact, a past release, a task — which PostgreSQL handles natively.

- **Phase 1:** Postgres full-text search (`tsvector` + GIN index) over contacts, releases, and generated assets, scoped to the band. Trigram indexes for fuzzy contact-name lookup.
- **Growth trigger:** a dedicated search engine (OpenSearch/Elasticsearch) becomes justified only if Bandly later adds genuine discovery — e.g., artists searching a shared venue/curator directory across bands. That is new product scope (see §22), not a scaling response to the current feature set.

§15

Notifications & Collaboration

Notifications

In-app notification center + transactional email (task due, booking follow-up due, DNA generation complete). No push notifications in Phase 1 — there's no native mobile app to push to. Every reminder traces back to a `Task` or `Interaction` row; nothing sends a notification that doesn't correspond to a concrete, dismissible item.

Collaboration, not messaging

The source material never describes chat between platform users — "communicatie" is about managing outbound email/WhatsApp *better*, and the band-coordination pain point is specifically "afspraken verdwijnen in groepsapps" (agreements get lost in group chats). The correct answer to that problem is a shared, assignable task list with lightweight comments — not a competing chat product. Real-time messaging is explicitly out of scope; see §22 if it's ever wanted.

§16

Payments & Billing

OPEN ARCHITECTURAL QUESTION

Neither document mentions pricing, subscription tiers, or any payment flow — between the platform and the artist, or between the artist and anyone else. This is a genuine gap, not an oversight in this blueprint.

The pragmatic default, consistent with "avoid unnecessary complexity": Stripe Billing for platform subscriptions (tiering likely against generation volume and band seats, given AI cost is the platform's real marginal cost — see §19), webhook-driven status sync into `bands.plan_tier`, no card data touches Bandly's own database. No user-to-user payment flow is designed, since there is no marketplace transaction for it to support.

§17

Infrastructure & Local Development

Containerized, cloud-agnostic: Next.js and Spring Boot as two container images behind a managed load balancer, managed PostgreSQL, managed object storage, managed Redis. No Kubernetes needed at this scale — a managed container platform (e.g., AWS ECS/Fargate, Fly.io, Render) is enough and keeps operational surface small.

FIG. 12 — LOCAL DOCKER-COMPOSE SERVICES

```
services:
  web        # Next.js dev server
  api        # Spring Boot, hot-reload
  postgres   # PostgreSQL 16
  minio      # S3-compatible storage emulator
  mailhog    # captures outbound email in dev
  redis      # optional — rate limiting / cache
```

Environments: **dev** (the compose file above), **staging** (same container images, seeded/anonymized data, real object storage bucket), **production** (managed Postgres with automated backups + PITR, WAF in front of the API, secrets in a managed secret store — never in environment files committed to the repo).

§18

CI/CD & Testing

Pipeline (GitHub Actions)

1. Lint + typecheck (ESLint/tsc for frontend, Checkstyle/Spotless for backend)
2. Unit + module tests (backend domain logic; frontend component tests)
3. Integration tests against Testcontainers PostgreSQL
4. OpenAPI contract check — generated client must match the live spec
5. Database migration check (Flyway) — dry-run against a fresh schema
6. Docker image build + push, tagged by commit SHA
7. Deploy to staging automatically; production on manual approval

Testing pyramid

LAYER	SCOPE	MANDATORY IN CI
Domain unit tests	Split-sheet percentage rules, checklist generation logic, DNA versioning	Yes
Repository/integration	Real Postgres via Testcontainers	Yes
API tests	Contract-verified against OpenAPI	Yes
Frontend component	React Testing Library on generation-review and checklist UI	Yes
End-to-end (Playwright)	Golden paths: onboarding → DNA, release → checklist, booking pitch → follow-up	On merge to main

§19

Observability & Scalability

Observability

- Structured JSON logs with a correlation ID threaded from the Next.js request through the Spring Boot call and any async job it spawns.
- Health/readiness endpoints for the container platform's own checks.
- Error tracking (Sentry) on both frontend and backend.
- **LLM spend as a first-class metric** — token usage and cost per band, tracked from `generation_jobs`, surfaced on an internal dashboard before it's ever surfaced to the artist. This is the platform's real variable cost and the one metric genuinely specific to this product.

Scalability — Day 1 vs. later

Day 1

Stateless Spring Boot instances behind a load balancer, HikariCP connection pooling, CDN for static/public media, single Postgres primary. This comfortably serves thousands of bands.

At meaningful scale

Postgres read replica for reporting/search queries, a separate worker pool for generation jobs (already isolated behind the outbox), Redis-backed caching for DNA/contact lookups, horizontal API scaling. Not designed in from day one — designed to be *addable* because of the module boundaries in §06.

§20

Architecture Decision Records

ADR-01

Modular monolith over microservices

ALTERNATIVES	Microservices per bounded context; a hybrid with only AI Generation extracted
REASONING	Single-tenant workflows are transactionally cohesive; team and load don't justify distributed complexity
CONSEQUENCES	Single deploy unit; module discipline in code review becomes the real enforcement mechanism
MIGRATION PATH	Extract AI Generation or Media first if their load profile (CPU/IO-bound, bursty) diverges from the rest — module boundaries make this a lift, not a redesign

ADR-02

REST over GraphQL

ALTERNATIVES	GraphQL for flexible client queries
REASONING	Band-scoped resource shapes are predictable; REST + OpenAPI gives typed client generation with far less server-side complexity
CONSEQUENCES	Occasional over/under-fetching on dashboard views, mitigated with purpose-built aggregate endpoints (e.g., `/focus`)
MIGRATION PATH	A GraphQL gateway could sit in front later without touching module internals

ADR-03

PostgreSQL as the only database

ALTERNATIVES	Postgres + Elasticsearch + Redis + a document store, from day one
REASONING	No requirement in the source material demands a second data store; Postgres full-text search and JSONB cover the flexible cases
CONSEQUENCES	Simpler ops, one backup/restore story
MIGRATION PATH	Add Redis for cache/rate-limit when contention appears; add a search engine only if real cross-band discovery ships

ADR-04

JWT access + httpOnly refresh cookie

ALTERNATIVES	Server-side session store; access token in localStorage
REASONING	Stateless access token keeps the API horizontally scalable; httpOnly refresh cookie avoids XSS token theft
CONSEQUENCES	Requires careful CSRF handling on the refresh endpoint (SameSite=Strict + origin check)
MIGRATION PATH	None needed — this scales as-is

ADR-05

Next.js App Router, mixed Server/Client Components

ALTERNATIVES	Fully client-rendered SPA
REASONING	Most Bandly views are read-heavy dashboards; Server Components remove client-side fetch waterfalls for them
CONSEQUENCES	Two mental models for data fetching; mitigated by scoping Client Components to genuinely interactive surfaces only
MIGRATION PATH	n/a

ADR-06

Server state via generated API client; no global client store

ALTERNATIVES	Redux/Zustand global store
REASONING	Almost all state is server-owned and band-scoped; a cache-on-fetch pattern covers it without a global store's overhead
CONSEQUENCES	Revisit if genuinely cross-cutting client-only state emerges (e.g., a real-time collab layer in §22)
MIGRATION PATH	Introduce a store scoped to that one feature, not app-wide

ADR-07

Postgres outbox instead of a message broker

ALTERNATIVES	Kafka, RabbitMQ, cloud-native queue
REASONING	Event volume and fan-out are small; the outbox gives durability and decoupling without new infrastructure

CONSEQUENCES	Poll-based dispatch has seconds-level latency, acceptable for reminders and checklist generation
MIGRATION PATH	Outbox table becomes the source Debezium/Kafka Connect would read from

ADR-08

Postgres full-text search over a dedicated search engine

ALTERNATIVES	OpenSearch/Elasticsearch from day one
REASONING	No cross-tenant discovery exists in the current scope; in-workspace lookup is well served by tsvector + trigram
CONSEQUENCES	Would need real re-architecture if a directory/discovery feature is added later — flagged explicitly in §22
MIGRATION PATH	Introduce OpenSearch alongside Postgres, sourced from the same outbox events

ADR-09

S3-compatible object storage from day one

ALTERNATIVES	Local/container disk storage for the MVP
REASONING	Audio masters, stems, and artwork are core to the product from the first release; retrofitting storage later means a painful data migration
CONSEQUENCES	Slightly more setup for local dev (MinIO), fully justified by avoiding a rewrite
MIGRATION PATH	n/a — this is the correct day-one choice

ADR-10

No real-time messaging layer in Phase 1

ALTERNATIVES	WebSocket-based in-app chat between band members
REASONING	Not described in the source material; the actual pain point (agreements lost in group chats) is solved by shared tasks, not chat
CONSEQUENCES	None for Phase 1 scope
MIGRATION PATH	WebSocket gateway is additive infrastructure if real-time collaboration is scoped in later (§22)

ADR-11

Managed container platform over Kubernetes

ALTERNATIVES	Self-managed Kubernetes cluster
REASONING	Two stateless services and a handful of managed data stores don't need Kubernetes' operational surface
CONSEQUENCES	Less portability across cloud providers than raw Kubernetes manifests
MIGRATION PATH	Container images are already the unit of deployment — moving to Kubernetes later is a platform swap, not an app rewrite

§21

MVP Architecture

MVP SCOPE all twelve source domains, one modular monolith, per your direction in §02.

- **Modules shipped:** Identity, Artist DNA, AI Generation kernel, Media, Planning, Content Studio, Release Manager, Rights & Admin, Relationships (booking + network), Finance, Grants, Notifications.
- **Infra:** one Next.js app, one Spring Boot app, managed Postgres, managed object storage, Redis for rate-limiting, Stripe for subscription billing.
- **External services:** one LLM provider (§11), transactional email provider, Google OAuth.
- **Explicitly not built:** distribution integrations, real-time collaboration, mobile apps, cross-tenant discovery, user-to-user payments.

§22

Growth Phase

GROWTH PHASE What the MVP architecture grows into — genuinely new capability, not withheld MVP scope.

FIG. 13 — MVP TO GROWTH-PHASE EVOLUTION



WHAT MUST BE CORRECT IN THE MVP TO MAKE THIS CHEAP

1. **Artist DNA as a versioned, first-class entity.** Retrofitting brand-consistency onto content generated without a DNA reference would mean re-generating a backlog of drafts.
2. **Band-scoped multi-tenancy from day one.** Adding multi-user organizations onto a single-user schema after the fact is a painful migration; starting band-scoped costs almost nothing extra now.
3. **Generated content as versioned, audited drafts.** Needed the moment compliance or legal review becomes a real requirement — much cheaper to have from the start than to backfill an audit trail.
4. **Object storage abstraction, not local disk.** Already ADR-09 — the expensive mistake to avoid.
5. **Outbox/event seams between modules.** Makes both service extraction and broker adoption additive rather than a re-plumbing exercise.

§23

Implementation Roadmap

FIG. 14 — MILESTONES

MILESTONE	SCOPE	DEFINITION OF DONE
M0 — Foundation	Repo scaffolding, CI/CD, auth, band creation/invite	A user can register, create a band, invite a member
M1 — Artist DNA	Identity profile intake, AI Generation kernel, DNA generation + versioning	A band has an approved, current DNA version
M2 — Planning & Notifications	Tasks, weekly focus view, outbox, reminder delivery	A manually created task sends a reminder on schedule
M3 — Release Manager & Media	Releases, tracks, object storage, checklist auto-generation	Creating a release produces a real, dated checklist with asset gaps flagged
M4 — Relationships	Contacts, interactions, bookings, AI-drafted pitches	A booking pitch is generated, sent, and its follow-up is reminded on time
M5 — Rights & Finance	Split sheets, contracts summaries, budgets, income	A track shows credits, rights status, and linked income together
M6 — Grants & Content Studio	Grant applications, social content generation	A grant application reuses DNA + an existing budget as generation context
M7 — Production hardening	Observability, security review, Stripe billing, GDPR export/delete	Ready for paying, non-pilot users
M8 — Growth phase	Per §22, prioritized against real usage data	Defined once M0–M7 usage data exists

§24

Risks, Open Questions & Final Recommendation

Risks worth designing against now

The product can become the pressure it's meant to relieve

Domain 12 is explicit: the system must not create more tasks, only clarity about which existing ones matter. Architecturally, this means the Planning and Notifications modules need a hard cap on visible items and daily notification volume — not a feature to add later, a constraint on the data model now.

AI-generated legal/financial content carries real liability

The source material flags this itself. Every generated asset in the Rights, Finance, and Grants domains carries a persistent "specialist review recommended" flag in the data model, and nothing in these domains auto-submits to a third party.

LLM cost is the platform's real marginal cost

Unlike most SaaS, Bandly's cost scales with usage in a way that's easy to underestimate. §19's cost-per-band metric and §08's per-band rate limiting exist specifically to keep this visible before it's a crisis.

Scope creep toward "marketplace"

Booking and network features read close to a two-sided marketplace. They aren't one in the source material — resist adding venue accounts, applications, or matching without treating it as new product scope with its own requirements pass.

Open questions to resolve before development starts

- Pricing/subscription model — no signal in the source material (§16)
- LLM provider and data-handling commitment (§11)
- Whether a user can belong to more than one band, and how that's priced
- Localization scope beyond Dutch/English
- Priority order for growth-phase distribution integrations (DistroKid vs. Spotify for Artists first)
- Whether band roles need more granularity than owner/admin/member

Final recommendation

If BANDLY starts development tomorrow: one Next.js app and one Spring Boot modular monolith, organized into the twelve modules in §06, backed by one PostgreSQL database with the schema in §07, deployed as two containers on a managed platform with managed Postgres, object storage, and Redis. Artist DNA is the first module built after auth, because every other module's value depends on it existing first. No message broker, no second database, no microservices, no marketplace — until real usage data, not this brief, asks for them.